UnixFS 파일 시스템

개요

UnixFS는 IPFS에서 파일과 디렉토리를 표현하기 위한 데이터 형식입니다. Unix 파일 시스템의 개념을 분산 환경에 적용하여, 파일의 메타데이터와 구조를 효율적으로 저장하고 관리할 수 있게 해줍니다. Boxo의 UnixFS 모듈은 파일 가져오기, 읽기, 쓰기, 디렉토리 관리 등의 핵심 기능을 제공합니다.

주요 개념

UnixFS 데이터 타입

UnixFS는 다음과 같은 데이터 타입을 지원합니다:

- **Raw**: 원시 바이트 데이터
- **File**: 일반 파일
- **Directory**: 디렉토리
- **Metadata**: 메타데이터
- **Symlink**: 심볼릭 링크
- **HAMTShard**: 대용량 디렉토리를 위한 샤드

DAG 구조

UnixFS는 Merkle DAG(Directed Acyclic Graph) 구조를 사용하여 파일과 디렉토리를 표현합니다:

```
파일 (큰 파일의 경우)
├── 청크 1
```

```
├── 청크 2
├── 청크 3
└── 청크 4

디렉토리
├── 파일 A
├── 파일 B
└── 하위 디렉토리
    ├── 파일 C
    └── 파일 D
```

기본 사용법

파일 가져오기 (Import)

```go
package main

import (
"context"
"fmt"
"strings"

"github.com/ipfs/boxo/chunker"
```

```
"github.com/ipfs/boxo/ipld/merkledag"
"github.com/ipfs/boxo/ipld/unixfs/importer"
"github.com/ipfs/go-datastore"
dssync  "github.com/ipfs/go-datastore/sync"
blockstore  "github.com/ipfs/go-ipfs-blockstore"
ipld  "github.com/ipfs/go-ipld-format"
)

func  importFile() error {
// 블록스토어와 DAG 서비스 설정
ds := dssync.MutexWrap(datastore.NewMapDatastore())
bs := blockstore.NewBlockstore(ds)
dagService := merkledag.NewDAGService(bs)

// 파일 내용 준비
content := "안녕하세요, IPFS UnixFS 예제입니다!"
reader := strings.NewReader(content)

// 청킹 전략 설정 (기본 크기 사용)
splitter := chunker.DefaultSplitter(reader)

// 파일을 DAG로 변환
node, err := importer.BuildDagFromReader(dagService, splitter)
```

```
if err != nil {
return fmt.Errorf("파일 가져오기 실패: %v", err)
}

fmt.Printf("파일이 성공적으로 가져와졌습니다. CID: %s\n", node.Cid())
return nil
}
```

파일 읽기

```
package main

import (
"context"
"fmt"
"io"

"github.com/ipfs/boxo/ipld/merkledag"
"github.com/ipfs/boxo/ipld/unixfs/io"
ipld "github.com/ipfs/go-ipld-format"
)

func readFile(dagService ipld.DAGService, node ipld.Node) error {
```

```
// UnixFS 파일 리더 생성
fileReader, err := uio.NewDagReader(context.Background(), node, dagService)
if err != nil {
return fmt.Errorf("파일 리더 생성 실패: %v", err)
}
defer fileReader.Close()

// 파일 내용 읽기
content, err := io.ReadAll(fileReader)
if err != nil {
return fmt.Errorf("파일 읽기 실패: %v", err)
}

fmt.Printf("파일 내용: %s\n", string(content))
return nil
}
```

청킹 전략과 설정

기본 청킹

```
// 기본 청크 크기 (256KB) 사용
```

```
splitter := chunker.DefaultSplitter(reader)
```

사용자 정의 청크 크기

```
// 1MB 청크 크기 설정
chunkSize := int64(1024 * 1024)
splitter := chunker.NewSizeSplitter(reader, chunkSize)
```

Rabin 청킹 (내용 기반)

```
// Rabin 청킹 사용 (중복 제거에 효과적)
splitter, err := chunker.NewRabin(reader, 1024*256) // 평균 256KB
if  err != nil {
return  err
}
```

디렉토리 처리

디렉토리 생성

```
package main

import (
"context"
```

```
"fmt"

"github.com/ipfs/boxo/ipld/merkledag"
"github.com/ipfs/boxo/ipld/unixfs"
"github.com/ipfs/boxo/ipld/unixfs/io"
ipld "github.com/ipfs/go-ipld-format"
)

func createDirectory(dagService ipld.DAGService) (ipld.Node, error) {
// 빈 디렉토리 생성
dirNode := unixfs.EmptyDirNode()
// DAG 서비스에 추가
err := dagService.Add(context.Background(), dirNode)
if err != nil {
return nil, fmt.Errorf("디렉토리 추가 실패: %v", err)
}

fmt.Printf("디렉토리가 생성되었습니다. CID: %s\n", dirNode.Cid())
return dirNode, nil
}
```

디렉토리에 파일 추가

```go
func addFileToDirectory(dagService ipld.DAGService, dirNode ipld.Node, fileName

// 디렉토리 편집기 생성

directory, err := uio.NewDirectoryFromNode(dagService, dirNode)

if err != nil {

return nil, fmt.Errorf("디렉토리 편집기 생성 실패: %v", err)

}



// 파일 추가

err = directory.AddChild(context.Background(), fileName, fileNode)

if err != nil {

return nil, fmt.Errorf("파일 추가 실패: %v", err)

}



// 업데이트된 디렉토리 노드 반환

updatedDir, err := directory.GetNode()

if err != nil {

return nil, fmt.Errorf("디렉토리 노드 가져오기 실패: %v", err)

}



return updatedDir, nil

}
```

디렉토리 내용 조회

```
func listDirectory(dagService ipld.DAGService, dirNode ipld.Node) error {
// 디렉토리 리더 생성
directory, err := uio.NewDirectoryFromNode(dagService, dirNode)
if err != nil {
return fmt.Errorf("디렉토리 리더 생성 실패: %v", err)
}

// 디렉토리 내용 순회
links := directory.Links(context.Background())
for link := range links {
if link.Err != nil {
return fmt.Errorf("링크 읽기 오류: %v", link.Err)
}
fmt.Printf("- %s (크기: %d, CID: %s)\n",
link.Link.Name, link.Link.Size, link.Link.Cid)
}

return nil
}
```

고급 활용

Trickle DAG 레이아웃

대용량 파일의 경우 Trickle 레이아웃을 사용하여 더 나은 성능을 얻을 수 있습니다:

```
// Trickle 레이아웃으로 파일 가져오기
node, err := importer.BuildTrickleDagFromReader(dagService, splitter)
if err != nil {
return fmt.Errorf("Trickle DAG 생성 실패: %v", err)
}
```

파일 시크(Seek) 기능

```
func seekInFile(dagService ipld.DAGService, node ipld.Node, offset int64) erro
// 파일 리더 생성
fileReader, err := uio.NewDagReader(context.Background(), node, dagService)
if err != nil {
return err
}
defer fileReader.Close()

// 특정 위치로 시크
_, err = fileReader.Seek(offset, io.SeekStart)
if err != nil {
return fmt.Errorf("시크 실패: %v", err)
```

```
}

// 해당 위치부터 데이터 읽기
buffer := make([]byte, 100)
n, err := fileReader.Read(buffer)
if err != nil && err != io.EOF {
return fmt.Errorf("읽기 실패: %v", err)
}

fmt.Printf("오프셋 %d부터 %d바이트 읽음: %s\n",
offset, n, string(buffer[:n]))
return nil
}
```

대용량 디렉토리 (HAMT)

많은 파일을 포함하는 디렉토리의 경우 HAMT(Hash Array Mapped Trie) 샤딩을 사용합니다:

```
// HAMT 샤딩이 자동으로 적용됩니다 (일반적으로 1000개 이상의 항목)
// 별도 설정 없이 기본 디렉토리 API 사용
```

설정 옵션

DagBuilderParams 설정

```
import "github.com/ipfs/boxo/ipld/unixfs/importer/helpers"


// 사용자 정의 DAG 빌더 매개변수
dbp := helpers.DagBuilderParams{
Dagserv: dagService,
Maxlinks: 1024, // 노드당 최대 링크 수
RawLeaves: true, // 리프 노드를 원시 형태로 저장
CidBuilder: nil, // 사용자 정의 CID 빌더
}


db, err := dbp.New(splitter)
if err != nil {
return err
}
```

모범 사례

1. 적절한 청크 크기 선택

```
// 작은 파일 (< 1MB): 기본 크기 사용
splitter := chunker.DefaultSplitter(reader)


// 큰 파일 (> 10MB): 더 큰 청크 크기 사용
```

```
splitter := chunker.NewSizeSplitter(reader, 1024*1024) // 1MB


// 중복 제거가 중요한 경우: Rabin 청킹 사용
splitter, _ := chunker.NewRabin(reader, 1024*256)
```

2. 메모리 효율적인 처리

```
// 큰 파일을 처리할 때는 스트리밍 방식 사용
func processLargeFile(reader io.Reader, dagService ipld.DAGService) error {
splitter := chunker.DefaultSplitter(reader)
// 메모리 사용량을 제한하기 위해 청크별로 처리
node, err := importer.BuildDagFromReader(dagService, splitter)
if err != nil {
return err
}
// 즉시 처리하여 메모리 해제
return processNode(node)
}
```

3. 오류 처리

```
func robustFileImport(reader io.Reader, dagService ipld.DAGService) (ipld.Node,
splitter := chunker.DefaultSplitter(reader)
```

```
node, err := importer.BuildDagFromReader(dagService, splitter)
if err != nil {
// 구체적인 오류 타입에 따른 처리
if errors.Is(err, context.DeadlineExceeded) {
return nil, fmt.Errorf("파일 가져오기 시간 초과: %w", err)
}
return nil, fmt.Errorf("파일 가져오기 실패: %w", err)
}
return node, nil
}
```

문제 해결

일반적인 오류와 해결책

1. **"malformed data in file format"**
   - 손상된 UnixFS 데이터를 읽으려 할 때 발생
   - 데이터 무결성 확인 및 재가져오기 필요
2. **"expected a ProtoNode as internal node"**
   - 잘못된 노드 타입을 사용할 때 발생
   - 올바른 UnixFS 노드 타입 사용 확인
3. **메모리 부족 오류**
   - 큰 파일 처리 시 발생

- 더 작은 청크 크기 사용 또는 스트리밍 처리 적용

성능 최적화

```
// 병렬 처리를 위한 설정
import "runtime"

func optimizedImport(reader io.Reader, dagService ipld.DAGService) (ipld.Node,
// CPU 코어 수에 따른 병렬 처리 설정
numWorkers := runtime.NumCPU()
// 적절한 청크 크기 설정
chunkSize := int64(256 * 1024) // 256KB
splitter := chunker.NewSizeSplitter(reader, chunkSize)
return importer.BuildDagFromReader(dagService, splitter)
}
```

UnixFS는 IPFS의 핵심 파일 시스템 구성 요소로, 효율적인 파일 저장과 검색을 가능하게 합니다. 적절한 청킹 전략과 설정을 통해 다양한 크기와 타입의 파일을 효과적으로 처리할 수 있습니다.